R PROGRAMMING
COMPLETE PRACTICAL CODEBOOK

All program listings • Copy-ready R code • Syllabus + notebook aligned

---

Course: Data Analysis Using R Programming Lab (25CSE221)

Companion to the R Programming Practical Master Guide

Included: vector and list programs, matrix operations, string processing, palindrome function, Titanic and Flight-price visualization, extra chart types named in the syllabus, exploratory data analysis, data manipulation, and web scraping.

How to use: Each numbered section is a complete code listing. Run one section at a time in RStudio. Programs that read a CSV use file.choose() so you can select the file on your own computer instead of editing a hard-coded Windows path.

Source alignment: The notebook and syllabus use different experiment numbering in places. The mapping page separates notebook programs from the formal syllabus items. Where the notebook has a clear coding issue or a machine-specific path, this codebook uses a runnable correction and notes the expected columns.

Program index and source mapping

Codebook section	Syllabus / notebook topic
1A–1D	Formal Experiment 1: vector filtering; list with vector/matrix/nested list; matrix arithmetic; matrix summaries and sorting. Notebook Experiments 1–4.
2A–2B	Formal Experiment 2: input string, count/uppercase/middle character; is_palindrome() function. Notebook Experiments 5–6.
3	Data visualization with Titanic CSV: bar, density plot, scatter, heatmap, interactivity, export. Notebook Experiment 7 / Visualization handout.
4	Data visualization with Flight-price CSV: bar, density, scatter, heatmap, interactive charts, export. Notebook Experiment 8.
5	Data analysis using Titanic data: summary, histogram, missing values, duplicate removal, imputation, filtering, sorting, boxplot, t-test, correlation. Notebook Experiment 9.
6	Data manipulation: read two files, merge by ID, aggregate by gender, filter, transform. Notebook Experiment 10.
7	Web scraping and data extraction: Books to Scrape, extract titles/prices, write CSV. Syllabus item 7 / notebook final program.
Extra chart patterns	Histogram, box plot, bar chart, line graph, scatter plot and pie chart listed in the syllabus.

Dataset rule: The notebook contains local paths that only work on the computer where they were written. file.choose() avoids that issue, but the selected CSV must still contain the column names required by the selected program.

0. Common package setup

Run the install command only once if these packages are not installed. For each new R session, run the library() commands you need. Base-R sections 1A–2B do not need extra packages.

```
# Install packages (run only once if not already installed)
install.packages(c(
  "ggplot2", "dplyr", "plotly", "htmlwidgets",
  "rvest", "xml2", "svglite"
))

# Load packages in a new R session when needed
library(ggplot2)
library(dplyr)
library(plotly)
library(htmlwidgets)

# Quick checks after importing any dataset
# names(my_data)   # list column names
# str(my_data)     # data types and structure
# head(my_data)    # first six rows
```

Important: R is case-sensitive. Survived and survived are different names. Check names(data) before running code that uses specific columns.

1A. Vector sequence and filtering

Task: Generate multiples of 5 from 5 to 60 and select values strictly greater than 15 and less than 45.

```
# Create multiples of 5 from 5 to 60
vector <- seq(5, 60, by = 5)
cat("Value of vector:", vector, "\n")

# Keep values greater than 15 AND less than 45
filtered_vector <- vector[vector > 15 & vector < 45]
cat("Elements greater than 15 and less than 45:",
    filtered_vector, "\n")
```

1B. List containing a vector, matrix and nested list

Task: Create the three kinds of objects, display the class of each list element, count outer-list objects and find the first vector length.

```
# Create a vector
my_vector <- c(1, 2, 3, 4, 5)

# Create a 3 x 3 matrix
my_matrix <- matrix(1:9, nrow = 3, ncol = 3)

# Create another (nested) list
my_nested_list <- list(
  sub_vector = c("a", "b", "c"),
  sub_number = 42
)

# Combine the vector, matrix and nested list
my_list <- list(
  vector = my_vector,
  matrix = my_matrix,
  nested_list = my_nested_list
)

# Display the class of each element
for (name in names(my_list)) {
  element_class <- class(my_list[[name]])
  cat("Class of", name, ":", element_class, "\n")
}

# Count objects in the outer list
cat("Number of objects in the list:", length(my_list), "\n")

# Length of the first vector
cat("Length of first vector:", length(my_list[[1]]), "\n")

# The same vector length using its name
cat("Length using $:", length(my_list$vector), "\n")
```

1C. Matrix addition, subtraction and product with transpose

Task: Use the matrix values written in the notebook. The product shown is matrix1 multiplied by the transpose of matrix1.

```
# Create matrix 1 (values from the notebook)
matrix1 <- matrix(
  c(2, 1, 1,
    1, 1, -1,
    1, 1, 2),
  nrow = 3, ncol = 3
)
print(matrix1)

# Create matrix 2 (values from the notebook)
matrix2 <- matrix(
  c(5, 2, 0,
    9, 3, 4,
    2, 3, 2),
  nrow = 3, ncol = 3
)
print(matrix2)

# Matrix addition
result <- matrix1 + matrix2
cat("Result of addition:\n")
print(result)

# Matrix subtraction
result <- matrix1 - matrix2
cat("Result of subtraction:\n")
print(result)

# Matrix multiplication: matrix1 times its transpose
result <- matrix1 %*% t(matrix1)
cat("Product of matrix1 and its transpose:\n")
print(result)
```

Matrix addition and subtraction operate element by element and require matching dimensions. %*% performs matrix multiplication; * is element-wise multiplication.

1D. Column sums, row means, total sum and column sorting

Task: Use matrix1 from Program 1C. This section is self-contained and recreates it.

```
# Recreate matrix1 from Program 1C
matrix1 <- matrix(
  c(2, 1, 1,
    1, 1, -1,
    1, 1, 2),
  nrow = 3, ncol = 3
)

# Sum each column
cat("Column sums:\n")
print(colSums(matrix1))

# Mean across each row
cat("Mean across rows:\n")
print(apply(matrix1, 1, mean))

# Total of all matrix elements
cat("Total sum:\n")
print(sum(matrix1))

# Sort each column in ascending order
cat("Matrix with each column sorted ascending:\n")
print(apply(matrix1, 2, sort))
```

2A. Input name, count characters, uppercase and middle character

Task: This follows the notebook approach of displaying one middle character using ceiling(n / 2). For an even-length string, it selects the left of the two central characters.

```
# Read the user's name
name <- readline(prompt = "Enter your name: ")

# Count the number of characters (spaces count too)
n <- nchar(name)

# Convert the name to uppercase
upper_name <- toupper(name)

# Find and extract the middle character
if (n == 0) {
  middle_char <- ""
  cat("The input is empty; there is no middle character.\n")
} else {
  mid <- ceiling(n / 2)
  middle_char <- substring(name, mid, mid)
}

# Display results
cat("Number of characters:", n, "\n")
cat("Uppercase:", upper_name, "\n")
if (n > 0) {
  cat("Middle character:", middle_char, "\n")
}
```

2B. is_palindrome() function

Task: Define a function with one parameter and determine whether its string reads the same forward and backward. This notebook-aligned version is case-sensitive and does not remove spaces or punctuation.

```
# Define a palindrome-checking function with one parameter
is_palindrome <- function(str) {
  chars <- strsplit(str, "")[[1]]
  reversed_chars <- rev(chars)
  reversed_str <- paste(reversed_chars, collapse = "")

  return(str == reversed_str)
}

# Get input and call the function
word <- readline(prompt = "Enter a string: ")
result <- is_palindrome(word)

if (result) {
  cat("The string is a palindrome.\n")
} else {
  cat("The string is not a palindrome.\n")
}

# Sample calls
is_palindrome("madam")
is_palindrome("hello")
```

3. Visualization program: Titanic dataset

Dataset requirements: The selected CSV must contain Pclass, Survived, Age and Fare. The following script includes the notebook's four chart types, a true line chart for syllabus coverage, interactive versions and exports.

```
# Required packages
library(ggplot2)
library(dplyr)
library(plotly)
library(htmlwidgets)

# Read the Titanic CSV file
# Select the Titanic CSV when the file picker opens
titanic <- read.csv(file.choose())

# Inspect the imported data
print(head(titanic))
str(titanic)
print(names(titanic))

# Check that the expected columns exist
required_columns <- c("Pclass", "Survived", "Age", "Fare")
if (!all(required_columns %in% names(titanic))) {
  stop("CSV must contain: Pclass, Survived, Age and Fare.")
}

# Treat survival status as a category
titanic$Survived <- factor(titanic$Survived, levels = c(0, 1))

# ------------------------------------------------
# A. BAR CHART: survival count by passenger class
# ------------------------------------------------
bar_chart <- ggplot(
  titanic,
  aes(x = factor(Pclass), fill = factor(Survived))
) +
  geom_bar(position = "dodge") +
  labs(
    title = "Survivors by Passenger Class",
    x = "Passenger Class",
    y = "Count",
    fill = "Survived"
  ) +
  scale_fill_manual(values = c("0" = "red", "1" = "blue")) +
  theme_minimal()
print(bar_chart)

# ------------------------------------------------
# B. DENSITY PLOT: age distribution
# The notebook calls this a line plot, but geom_density()
# actually creates a density plot.
# ------------------------------------------------
density_plot <- ggplot(titanic, aes(x = Age)) +
  geom_density(fill = "blue", alpha = 0.5, na.rm = TRUE) +
  labs(
    title = "Age Distribution of Passengers",
    x = "Age",
    y = "Density"
  ) +
  theme_minimal()
print(density_plot)

# ------------------------------------------------
# C. SCATTER PLOT: age versus fare, colour by survival
# ------------------------------------------------
scatter_plot <- ggplot(
  titanic,
  aes(x = Age, y = Fare, color = Survived)
) +
  geom_point(na.rm = TRUE, alpha = 0.7) +
  labs(
    title = "Fare versus Age",
    x = "Age",
    y = "Fare",
    color = "Survived"
  ) +
  scale_color_manual(values = c("0" = "red", "1" = "steelblue")) +
  theme_minimal()
print(scatter_plot)

# ------------------------------------------------
# D. CATEGORICAL HEATMAP: passenger class vs survival
```

```
# Count first, then draw tiles. This is clearer for categories
# than using a two-dimensional numeric binning function.
# ------------------------------------------------
heat_counts <- titanic %>%
  filter(!is.na(Pclass), !is.na(Survived)) %>%
  count(Pclass, Survived, name = "Frequency")

heatmap_plot <- ggplot(
  heat_counts,
  aes(x = factor(Pclass), y = Survived, fill = Frequency)
) +
  geom_tile(color = "white") +
  labs(
    title = "Titanic Survival Heatmap",
    x = "Passenger Class",
    y = "Survived (0 = No, 1 = Yes)",
    fill = "Frequency"
  ) +
  scale_fill_gradient(low = "white", high = "blue") +
  theme_minimal()
print(heatmap_plot)

# ------------------------------------------------
# E. TRUE LINE CHART: average fare by age group
# ------------------------------------------------
titanic$AgeGroup <- cut(
  titanic$Age,
  breaks = c(0, 12, 18, 30, 45, 60, Inf),
  right = FALSE
)

fare_by_age <- titanic %>%
  filter(!is.na(AgeGroup), !is.na(Fare)) %>%
  group_by(AgeGroup) %>%
  summarise(MeanFare = mean(Fare), .groups = "drop")

line_chart <- ggplot(
  fare_by_age,
  aes(x = AgeGroup, y = MeanFare, group = 1)
) +
  geom_line() +
  geom_point() +
  labs(
    title = "Average Fare by Age Group",
    x = "Age Group",
    y = "Average Fare"
  ) +
  theme_minimal()
print(line_chart)

# ------------------------------------------------
# F. INTERACTIVE CHARTS: hover, zoom and legend controls
# ------------------------------------------------
interactive_bar <- ggplotly(bar_chart)
interactive_density <- ggplotly(density_plot)
interactive_scatter <- ggplotly(scatter_plot)
interactive_heatmap <- ggplotly(heatmap_plot)
interactive_line <- ggplotly(line_chart)

print(interactive_bar)
print(interactive_density)
print(interactive_scatter)
print(interactive_heatmap)
print(interactive_line)

# ------------------------------------------------
# G. EXPORT STATIC CHARTS
# Files are saved in the current working directory.
# ------------------------------------------------
ggsave("titanic_bar.png", plot = bar_chart,
       width = 8, height = 6, dpi = 300)
ggsave("titanic_density.jpg", plot = density_plot,
       width = 8, height = 6, dpi = 300)
ggsave("titanic_scatter.png", plot = scatter_plot,
       width = 8, height = 6, dpi = 300)
ggsave("titanic_heatmap.pdf", plot = heatmap_plot,
       width = 8, height = 6)
ggsave("titanic_line.svg", plot = line_chart,
       width = 8, height = 6)

# ------------------------------------------------
# H. EXPORT AN INTERACTIVE CHART TO HTML
# selfcontained = TRUE may require Pandoc installed.
```

```
# ------------------------------------------------
saveWidget(
  interactive_bar,
  file = "titanic_interactive_bar.html",
  selfcontained = TRUE
)
```

The four original notebook chart types are bar, density-style plot, scatter and heatmap. A separate true line chart is included because the syllabus names line graphs; a density curve is not technically a line chart.

4. Visualization program: Flight-price dataset

Dataset requirements: The selected CSV must contain Airline, Total_Stops, Source and Price. If your file uses different column names, check names(flight_data) and update the corresponding names in the code.

```
library(ggplot2)
library(dplyr)
library(plotly)
library(htmlwidgets)

# Select the Flight-price CSV
flight_data <- read.csv(file.choose())

# Inspect the data
print(head(flight_data))
str(flight_data)
print(names(flight_data))

required_columns <- c("Airline", "Total_Stops", "Source", "Price")
if (!all(required_columns %in% names(flight_data))) {
  stop("CSV must contain Airline, Total_Stops, Source and Price.")
}

flight_data$Airline <- as.factor(flight_data$Airline)
flight_data$Total_Stops <- as.factor(flight_data$Total_Stops)
flight_data$Source <- as.factor(flight_data$Source)

# ------------------------------------------------
# A. BAR CHART: flight count by airline and number of stops
# ------------------------------------------------
bar_chart <- ggplot(
  flight_data,
  aes(x = Airline, fill = Total_Stops)
) +
  geom_bar(position = "dodge") +
  labs(
    title = "Flight Count by Airline",
    x = "Airline",
    y = "Count",
    fill = "Total Stops"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
print(bar_chart)

# ------------------------------------------------
# B. DENSITY PLOT: ticket-price distribution
# ------------------------------------------------
density_plot <- ggplot(flight_data, aes(x = Price)) +
  geom_density(fill = "green", alpha = 0.5, na.rm = TRUE) +
  labs(
    title = "Price Distribution of Flights",
    x = "Ticket Price",
    y = "Density"
  ) +
  theme_minimal()
print(density_plot)

# ------------------------------------------------
# C. SCATTER PLOT: price by airline, coloured by total stops
# ------------------------------------------------
scatter_plot <- ggplot(
  flight_data,
  aes(x = Airline, y = Price, color = Total_Stops)
) +
  geom_jitter(width = 0.2, alpha = 0.6, na.rm = TRUE) +
  labs(
    title = "Flight Price versus Airline",
    x = "Airline",
    y = "Price",
    color = "Total Stops"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
print(scatter_plot)

# ------------------------------------------------
# D. CATEGORICAL HEATMAP: frequency by airline and source
# ------------------------------------------------
heat_counts <- flight_data %>%
  filter(!is.na(Airline), !is.na(Source)) %>%
  count(Airline, Source, name = "Frequency")
```

```
heatmap_plot <- ggplot(
  heat_counts,
  aes(x = Airline, y = Source, fill = Frequency)
) +
  geom_tile(color = "white") +
  labs(
    title = "Flight Frequency Heatmap",
    x = "Airline",
    y = "Source",
    fill = "Frequency"
  ) +
  scale_fill_gradient(low = "white", high = "darkgreen") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
print(heatmap_plot)

# ------------------------------------------------
# E. INTERACTIVE CHARTS
# ------------------------------------------------
interactive_bar <- ggplotly(bar_chart)
interactive_density <- ggplotly(density_plot)
interactive_scatter <- ggplotly(scatter_plot)
interactive_heatmap <- ggplotly(heatmap_plot)

print(interactive_bar)
print(interactive_density)
print(interactive_scatter)
print(interactive_heatmap)

# ------------------------------------------------
# F. EXPORT STATIC CHARTS
# ------------------------------------------------
ggsave("flight_bar.png", plot = bar_chart,
       width = 9, height = 6, dpi = 300)
ggsave("flight_density.jpg", plot = density_plot,
       width = 8, height = 6, dpi = 300)
ggsave("flight_scatter.png", plot = scatter_plot,
       width = 9, height = 6, dpi = 300)
ggsave("flight_heatmap.pdf", plot = heatmap_plot,
       width = 9, height = 6)

# Export one interactive chart to HTML
saveWidget(
  interactive_bar,
  file = "flight_interactive_bar.html",
  selfcontained = TRUE
)
```

4A. Extra chart patterns listed in the syllabus

Use this as a separate quick-reference program for histogram, box plot, bar chart, line graph, scatter plot and pie chart. It uses the Titanic dataset loaded with file.choose().

```
library(ggplot2)

# Read CSV and inspect column names
titanic <- read.csv(file.choose())
print(names(titanic))

# 1. HISTOGRAM: distribution of passenger ages
hist(
  titanic$Age[!is.na(titanic$Age)],
  main = "Age Distribution",
  xlab = "Age",
  ylab = "Frequency",
  col = "lightblue",
  border = "white"
)

# 2. BOX PLOT: fare by passenger class
boxplot(
  Fare ~ Pclass,
  data = titanic,
  main = "Fare by Passenger Class",
  xlab = "Passenger Class",
  ylab = "Fare",
  col = c("lightblue", "lightgreen", "pink")
)

# 3. BAR CHART: passenger count in each class
barplot(
  table(titanic$Pclass),
  main = "Passengers by Class",
  xlab = "Passenger Class",
  ylab = "Count",
  col = "steelblue"
)

# 4. LINE GRAPH: example sequence over five points
x <- 1:5
y <- c(3, 5, 4, 7, 9)
plot(
  x, y, type = "o",
  main = "Example Line Graph",
  xlab = "X",
  ylab = "Y",
  col = "darkgreen"
)

# 5. SCATTER PLOT: age versus fare
plot(
  titanic$Age, titanic$Fare,
  main = "Fare versus Age",
  xlab = "Age",
  ylab = "Fare",
  pch = 19,
  col = "steelblue"
)

# 6. PIE CHART: survival counts
survival_counts <- table(titanic$Survived)
pie(
  survival_counts,
  main = "Survival Counts",
  labels = names(survival_counts),
  col = c("tomato", "seagreen")
)
```

5. Data analysis program: Titanic dataset

Dataset requirements: This program expects a Titanic CSV with Age, Fare and Survived columns. Missing-value and summary outputs depend on the actual dataset, so no fixed numeric output is printed in this codebook.

```
library(dplyr)
library(ggplot2)

# Read a Titanic CSV
data <- read.csv(file.choose())
print(head(data))
print(names(data))

# A. Exploratory summary
summary_status <- summary(data)
print(summary_status)

# B. Histogram of age
histogram <- ggplot(data, aes(x = Age)) +
  geom_histogram(
    binwidth = 5,
    fill = "black",
    color = "green",
    na.rm = TRUE
  ) +
  labs(
    title = "Distribution of Age on Titanic",
    x = "Age",
    y = "Frequency"
  ) +
  theme_minimal()
print(histogram)

# C. Count missing values in each column
missing_values <- colSums(is.na(data))
print(missing_values)

# D. Remove duplicate rows
data <- distinct(data)

# E. Replace missing Age values with the mean Age
age_mean <- mean(data$Age, na.rm = TRUE)
if (is.finite(age_mean)) {
  data$Age[is.na(data$Age)] <- age_mean
}

# Check remaining missing values
missing_after_cleaning <- colSums(is.na(data))
print(missing_after_cleaning)

# F. Filter passengers older than 18
adult_passengers <- filter(data, Age > 18)
print(head(adult_passengers))

# G. Sort by Fare in descending order
sorted_data <- arrange(data, desc(Fare))
print(head(sorted_data))

# H. Box plot: Age by survival status
box_plot <- ggplot(
  data,
  aes(x = factor(Survived), y = Age, fill = factor(Survived))
) +
  geom_boxplot(na.rm = TRUE) +
  labs(
    title = "Box Plot of Age by Survival Status",
    x = "Survived (0 = No, 1 = Yes)",
    y = "Age",
    fill = "Survived"
  ) +
  theme_minimal()
print(box_plot)

# I. Independent two-sample t-test for age by survival status
# Requires two survival groups in the selected CSV.
t_test_result <- t.test(Age ~ Survived, data = data)
print(t_test_result)

# J. Correlation between Age and Fare
correlation_coefficient <- cor(
  data$Age,
  data$Fare,
```

```
  use = "complete.obs"
)
print(correlation_coefficient)

# K. Bar chart of survival counts
survival_bar <- ggplot(
  data,
  aes(x = factor(Survived), fill = factor(Survived))
) +
  geom_bar() +
  labs(
    title = "Number of Survivors on Titanic",
    x = "Survived (0 = No, 1 = Yes)",
    y = "Count",
    fill = "Survived"
  ) +
  scale_fill_manual(values = c("0" = "red", "1" = "green")) +
  theme_minimal()
print(survival_bar)

# L. Save cleaned data
write.csv(data, "titanic_cleaned.csv", row.names = FALSE)
```

The notebook also shows merging two Titanic files when applicable. If your practical requires that step, run the next independent block and replace the key name only if your actual CSV uses a different common ID column.

5A. Optional: merge two datasets by PassengerId

```
# Select the first and second CSV files in the file picker
main_data <- read.csv(file.choose())
second_data <- read.csv(file.choose())

# Inspect names before joining
print(names(main_data))
print(names(second_data))

merge_key <- "PassengerId"
if (!(merge_key %in% names(main_data)) ||
    !(merge_key %in% names(second_data))) {
  stop("Both files must contain the selected common key column.")
}

merged_titanic <- merge(
  main_data,
  second_data,
  by = merge_key,
  suffixes = c("_main", "_second")
)
print(head(merged_titanic))
```

6. Data manipulation program: merge, aggregate, filter and transform

Dataset requirements: Select two CSV files that share an ID column. After joining, the data must have Gender, Salary and Age columns for the aggregation and transformation steps below. If your instructor uses a different common key, change by = "ID" to that exact column name.

```
library(dplyr)

# Select the first and second dataset CSVs
dataset1 <- read.csv(file.choose())
dataset2 <- read.csv(file.choose())

# Inspect their column names
print(names(dataset1))
print(names(dataset2))

# Check that both files share ID
if (!("ID" %in% names(dataset1)) || !("ID" %in% names(dataset2))) {
  stop("Both datasets must contain an ID column for this join.")
}

# Merge datasets using the common key
merged_data <- merge(dataset1, dataset2, by = "ID")
cat("Merged data:\n")
print(head(merged_data))

# Confirm the columns needed for the next steps
needed_columns <- c("Gender", "Salary", "Age")
if (!all(needed_columns %in% names(merged_data))) {
  stop("Merged data must contain Gender, Salary and Age columns.")
}

# Aggregate by gender: total salary, average age and record count
aggregated_data <- merged_data %>%
  group_by(Gender) %>%
  summarise(
    total_salary = sum(Salary, na.rm = TRUE),
    average_age = mean(Age, na.rm = TRUE),
    count = n(),
    .groups = "drop"
  )
cat("Aggregated data:\n")
print(aggregated_data)

# Filter records where Age is greater than 25
filtered_data <- merged_data %>%
  filter(Age > 25)
cat("Filtered data (Age > 25):\n")
print(head(filtered_data))

# Add a new column that doubles Salary
transformed_data <- merged_data %>%
  mutate(double_salary = Salary * 2)
cat("Transformed data:\n")
print(head(transformed_data))

# Export results
write.csv(merged_data, "merged_data.csv", row.names = FALSE)
write.csv(aggregated_data, "aggregated_data.csv", row.names = FALSE)
write.csv(filtered_data, "filtered_data.csv", row.names = FALSE)
write.csv(transformed_data, "transformed_data.csv", row.names = FALSE)
```

7. Web scraping and data extraction

Task: Retrieve an HTML page, extract book titles and prices using XPath selectors, create a data frame, and save the extracted data to CSV. This follows the book website used in the notebook.

```
# Install once if needed
install.packages(c("rvest", "xml2"))

library(rvest)

# Define the target website
url <- "https://books.toscrape.com/"

# Read the HTML content
html_content <- read_html(url, encoding = "UTF-8")

# XPath selectors for book titles and prices
# Each book is inside an article with class product_pod
title_xpath <- "//article[contains(@class, 'product_pod')]/h3/a"
price_xpath <- paste0(
  "//article[contains(@class, 'product_pod')]",
  "//p[contains(@class, 'price_color')]"
)

# Extract book titles from the title attribute
title_nodes <- html_elements(html_content, xpath = title_xpath)
titles <- html_attr(title_nodes, "title")

# Extract displayed prices
price_nodes <- html_elements(html_content, xpath = price_xpath)
prices <- html_text2(price_nodes)

# Combine the scraped fields into a data frame
book_data <- data.frame(
  Title = titles,
  Price = prices,
  stringsAsFactors = FALSE
)

# View the results and save them
print(head(book_data))
write.csv(book_data, "books_scraped.csv", row.names = FALSE)

cat("Books extracted:", nrow(book_data), "\n")
cat("Saved file: books_scraped.csv\n")
```

Web scraping requires an internet connection and the target site to permit access. Use it for learning and respect the website’s terms and access rules.

Final copy-and-run checklist

- Run one program at a time; do not paste every experiment into one session without checking variable names.
- Install optional packages once; load them using library() in each fresh R session.
- Use file.choose() to pick the correct CSV when a program requests a file.
- Run names(data) and str(data) first when a plot or analysis program expects specific columns.
- Titanic visualization requires Pclass, Survived, Age and Fare.
- Flight visualization requires Airline, Total_Stops, Source and Price.
- Data manipulation requires two CSVs with a shared ID key; the merged data needs Gender, Salary and Age.
- Web scraping requires internet access and the rvest and xml2 packages.
- Static plot exports appear in getwd(), the current working directory. Check it with getwd().
- If an error says object not found, check spelling and capitalization with names(data).

Practical advice: Memorize the program structure and the purpose of each key function, not just the colours. In viva, explain data source, mapping (aes), geometry (geom_*), labels (labs), and output/export functions.